

Prise en main du langage python (TP 2)

Modules

Pour importer des modules, on utilise l'une des syntaxes suivantes :

```
from nom_module import nom_fonction(s)
```

```
import nom_module
```

```
import nom_module as nom_court
```

1 - Numpy

```
In [1]: import numpy as np
```

Numpy contient des outils de manipulation de tableaux, de matrices, et fonctions mathématiques (racine, exp, cos ... elles sont aussi dans le module math)

```
In [2]: print(np.sqrt(4))
```

```
In [3]: print(np.cos(np.pi/2))  
print(np.exp(5))
```

Numpy permet de faire des calculs sur des tableaux : array

```
In [4]: liste = [1,3,4,5]  
print(liste)  
tab = np.array(liste)  
print(tab)  
print(type(liste), type(tab))
```

```
In [5]: print(tab[0])  
print(tab[1:3])
```

```
In [6]: tab2d = np.array([ [1,2], [3,4], [5,6] ])

print(tab2d)
print(len(tab2d))
print(tab2d.ndim)
print(tab2d.shape)
```

```
In [7]: print(tab2d[0,0])
print(tab2d[0,:])
print(tab2d[:,1])
```

```
In [8]: x = np.arange(0,2.1,0.1)
print(x)
print(type(x))
```

```
In [9]: y = np.linspace(0,2, 21)
print(y)
print(type(y))
```

```
In [10]: print(y[2])
```

Tableaux 2d = matrices

```
In [11]: A = np.array([ [1,2,3] , [4,5,6] , [6,7,8] ])
print(A)
```

```
In [12]: B = 2*A  
print(B)
```

```
In [13]: C = A+B  
print(C)
```

```
In [14]: print(A*B)
```

```
In [15]:  
print(np.dot(A,B))  
print(A@B)
```

```
In [16]: v = np.array([0,1,2])  
print(v)  
print(np.dot(A,v))
```

```
In [17]: print(A**2)  
print(np.dot(A,A))
```

```
In [18]: print(np.dot(v,A))
```

```
In [19]: print(A)
          print(A.T)
          print(A.transpose())
```

Quelques matrices particulières

```
In [20]: print(np.diag([1,3,5]))
```

```
In [21]: print(np.zeros((2,3)))
          print(np.ones((3,2)))
```

```
In [22]: print(np.eye(2,5))
```

```
In [23]: print(np.random.rand(2,4))
```

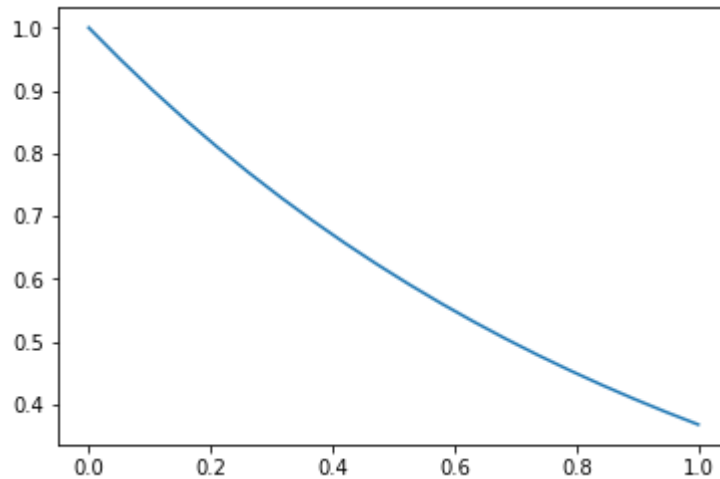
Exercice : Calculer le produit $\begin{pmatrix} 1 & 2 & 3 \\ 1 & -2 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \\ 7 \end{pmatrix}$

2 - Matplotlib

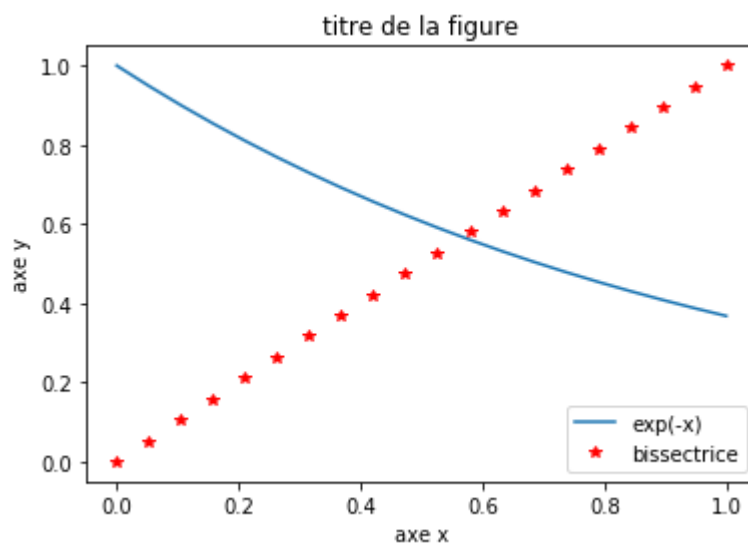
```
In [25]: import matplotlib.pyplot as plt
```

Bibliothèque pour effectuer des graphiques

```
In [26]: x = np.linspace(0,1, 20)
y = np.exp(-x)
plt.figure()
plt.plot(x,y)
plt.show()
```

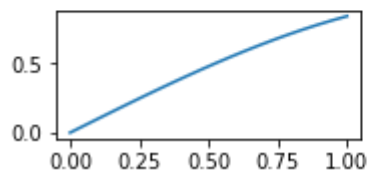
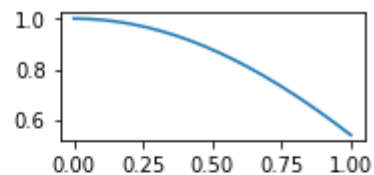


```
In [27]: plt.figure()
plt.plot(x,y, label='exp(-x)')
plt.plot(x,x, 'r *', label='bissectrice')
plt.title('titre de la figure')
plt.xlabel('axe x')
plt.ylabel('axe y')
plt.legend()
plt.show()
```

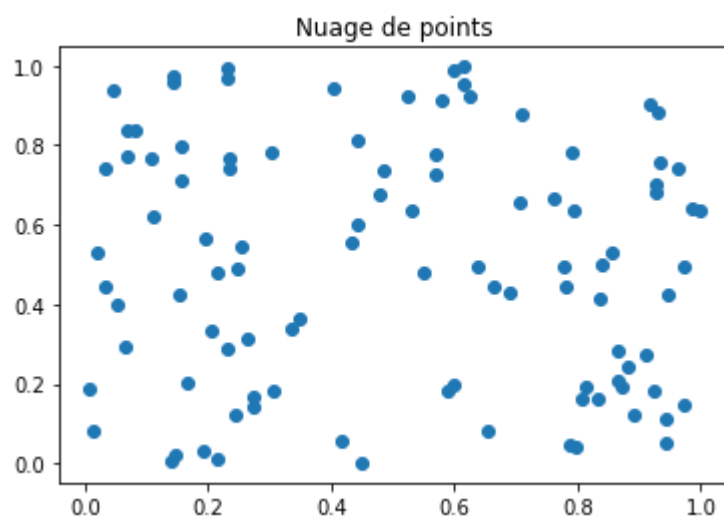


```
In [28]: t = np.linspace(0,1,20)
y1 = np.cos(t)
y2 = np.sin(t)
plt.figure()
plt.subplot(3,2,2)

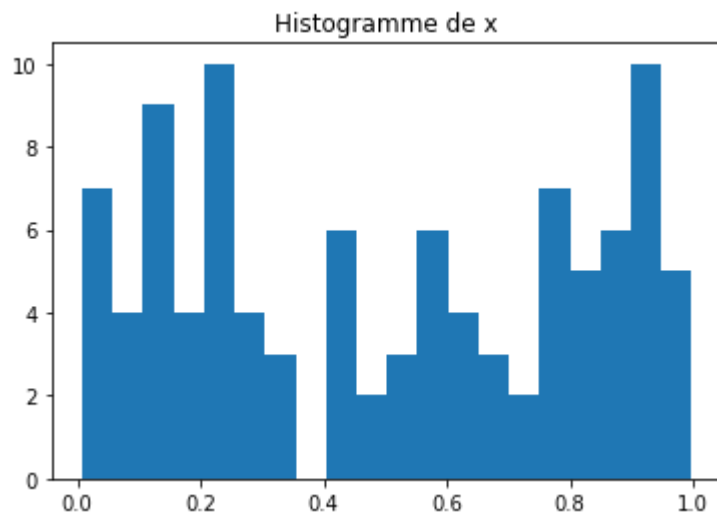
plt.plot(t,y1)
plt.subplot(3,2,5)
plt.plot(t,y2)
plt.show()
```



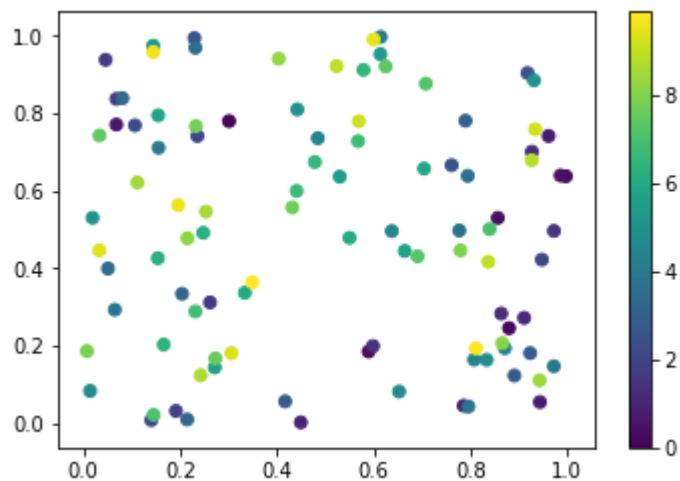
```
In [29]: Npts = 100
x = np.random.rand(Npts, 1)
y = np.random.rand(Npts, 1)
plt.figure()
plt.title('Nuage de points')
plt.plot(x,y, 'o')
plt.show()
```



```
In [30]: Nbins = 20
plt.figure()
plt.hist(x, bins=Nbins)
plt.title('Histogramme de x')
plt.show()
```



```
In [31]: plt.figure()
z = np.array([np.arange(0,10,0.1)]).T
plt.scatter(x,y, c=z)
plt.colorbar()
plt.show()
```



```
In [32]: z = np.array([np.arange(0,10,0.1)]).T
print(x[0,0], y[0,0], z[0,0])
```

3 - Scipy

bibliothèque d'outils numériques pour les tableaux de numpy

Optimisation

```
In [33]: import scipy.optimize as op
```

```
In [34]: def g(x) :
          return 5*x**6 + 2*x-1
          # Exercice : tracer g sur [-2,1]
```

```
In [35]: print(op.fmin(g, -0.5))
```

```
In [36]: print(op.fsolve(g, 0.5))
```

Algèbre linéaire

```
In [37]: import scipy.linalg as la
```

```
In [38]: A = np.array([ [0,1,1] , [1,0,1], [1,1,0]])
          B = la.inv(A)
          print(A)
          print(B)
          print(A@B)
```

```
In [39]: b = np.array([5,4,3])
          x = la.solve(A,b)
          print('x=',x)
          print(A@x)
```

Integration


```
In [40]: import scipy.integrate as integ
```

calcul d'intégrales ou résolution d'edo